



Problem Solving With C
UE24CS151B

By,
Prof. Bivas Bhattacharya,
Assistant Professor

Question 1:

- **Question:** Explain the Program Development Life Cycle (PDLC). Describe at least four phases involved, their purpose, and how they contribute to successful software development.
- **Answer:**
- The Program Development Life Cycle (PDLC) is a structured process used to plan, design, create, test, and deploy information systems. It provides a framework for managing the complexity of software development.
- **Requirements Gathering:** This phase involves identifying and documenting the needs and expectations of the users and stakeholders. It defines *what* the software should do. Properly gathered requirements prevent developing features that aren't needed.
- **Design:** This phase translates the requirements into a detailed plan or blueprint for the software. It specifies *how* the software will meet the requirements, including data structures, algorithms, user interfaces, and system architecture. A good design makes implementation smoother and more maintainable.
- **Implementation (Coding):** This is where the actual code is written based on the design specifications. It involves choosing the appropriate programming language and tools to translate the design into working software.
- **Testing:** This phase involves verifying that the software functions correctly and meets the specified requirements. Different types of testing are performed, such as unit testing, integration testing, system testing, and user acceptance testing. Thorough testing is crucial for identifying and fixing bugs before deployment.
- (Other phases could be Deployment and Maintenance)

Question 2:

- **Question:** Describe the different ways to compile and execute a C program using GCC, as presented in the document. Explain the purpose of each step involved in each approach.
- **Answer:**
- The document outlines three main ways to compile and execute C code using GCC.
- **Way 1: Simple Compilation:** `gcc <filename.c>` followed by `./a.out` (or `a.exe`). This is the simplest approach. The `gcc <filename.c>` command compiles and links the C source code into an executable file named `a.out` (on Linux/macOS) or `a.exe` (on Windows). The `./a.out` (or `a.exe`) command then executes the compiled program. This is quick, but doesn't allow for separate compilation of multiple files or linking with specific libraries in a controlled manner.
- **Way 2: Separate Compilation and Linking:** `gcc -c <filename.c>` followed by `gcc <filename.o>` (or `gcc <list of object files> -l<library>`). This approach separates the compilation and linking steps. `gcc -c <filename.c>` compiles the C source code into an object file (`.o`) without linking. The `gcc <filename.o>` command then links the object file(s) to create the executable. The `-l<library>` option is used to link against external libraries. This method is useful for larger projects with multiple source files, as it allows you to recompile only the files that have changed.
- **Way 3: Renaming the Output File:** `gcc <filename.c> -o <imagename>` followed by `<imagename>`. This approach combines compilation and linking, but allows you to specify the name of the output executable file. The `gcc <filename.c> -o <imagename>` command compiles and links the C source code into an executable file named `<imagename>`. This is useful for creating executables with meaningful names.

Question 3:

- **Question:** Explain the significance of C compiler standards (like C89, C99, and C11). Why are they important, and how can you specify a particular standard when compiling with GCC? Also, how can you check the C standard version at compile time?
- **Answer:**
- C compiler standards (C89, C99, C11, etc.) are important because they provide a consistent and well-defined set of rules for the C language.
- **Importance:**
- **Portability:** Standards ensure that code written according to the standard will compile and run correctly on different platforms with conforming compilers.
- **Consistency:** Standards promote consistency in code style and behavior, making it easier for developers to understand and maintain code written by others.

- **Evolution:** Standards allow the C language to evolve over time, adding new features and addressing limitations, while maintaining backward compatibility as much as possible.
- **Specifying the standard:** The `-std=` option in GCC allows you to specify the C standard to use during compilation. For example, `gcc -std=c99 program.c` compiles the `program.c` file using the C99 standard.
- **Checking at compile time:** The `__STDC_VERSION__` macro is a predefined macro that expands to a long integer constant that indicates the version of the C standard being used. You can use this macro in your code with `#ifdef` to conditionally compile code based on the C standard.

Question 4:

- **Question:** Describe the four types of errors that can occur during the execution of a C program, as mentioned in the presentation. Provide a specific example of each type of error.
- **Answer:**
- The presentation identifies four main categories of errors that can occur during C program execution: Compile-time errors, Link-time errors, Run-time errors, and Logical errors.
- **Compile-time Error:** These errors occur during the compilation phase. They are usually due to syntax errors or violations of the C language rules.
- **Example:** `int x = ;` (Missing an expression after the equals sign). The compiler will flag this as a syntax error.
- **Link-time Error:** These errors occur during the linking phase, when the object code is combined to create the executable. They usually involve missing or undefined functions or variables.
- **Example:** Calling a function `my_function()` in your code, but `my_function()` is not defined in any of your source files or linked libraries. The linker will report an "undefined reference" error.
- **Run-time Error:** These errors occur while the program is running. They are often caused by unexpected or invalid operations.
- **Example:** `int result = 10 / 0;` (Division by zero). This will cause a run-time error, typically resulting in program termination.
- **Logical Error:** These errors occur when the program compiles and runs without crashing, but produces incorrect results due to flaws in the program's logic.
- **Example:** Calculating the average of two numbers by adding them but then multiplying by 0.5 instead of dividing by 2: `float average = (a + b) * 0.5;`. The program will run, but the average will be incorrect.

Question 5:

- **Question:** Suppose you have a C program consisting of multiple source code files (e.g., `main.c`, `functions.c`, and `utils.c`) and you want to create an executable called "myprogram". Detail the steps you would take using GCC to compile and link

these files, assuming `functions.c` depends on a library called "mylib". Include the specific GCC commands you would use.

- **Answer:**
- To compile and link multiple C source files using GCC, you would typically use a combination of the `-c` (compile) and linking steps.
- **Step 1: Compile each source file into an object file:**
- `gcc -c main.c` (creates `main.o`)
- `gcc -c functions.c` (creates `functions.o`)
- `gcc -c utils.c` (creates `utils.o`)
- **Step 2: Link the object files together to create the executable, including the "mylib" library:**
- `gcc main.o functions.o utils.o -o myprogram -lmylib`
- `-o myprogram` specifies the name of the output executable file as "myprogram."
- `-lmylib` tells the linker to include the "mylib" library. Note that the linker will search for a file named `libmylib.a` or `libmylib.so` (depending on whether it's a static or shared library) in the standard library search paths. If the library is in a non-standard location, you might need to add the `-L` option to specify the library's directory (e.g., `-L/path/to/library`).